

04. Image Pyramids

Jae-Young Choi

Sungkyunkwan University

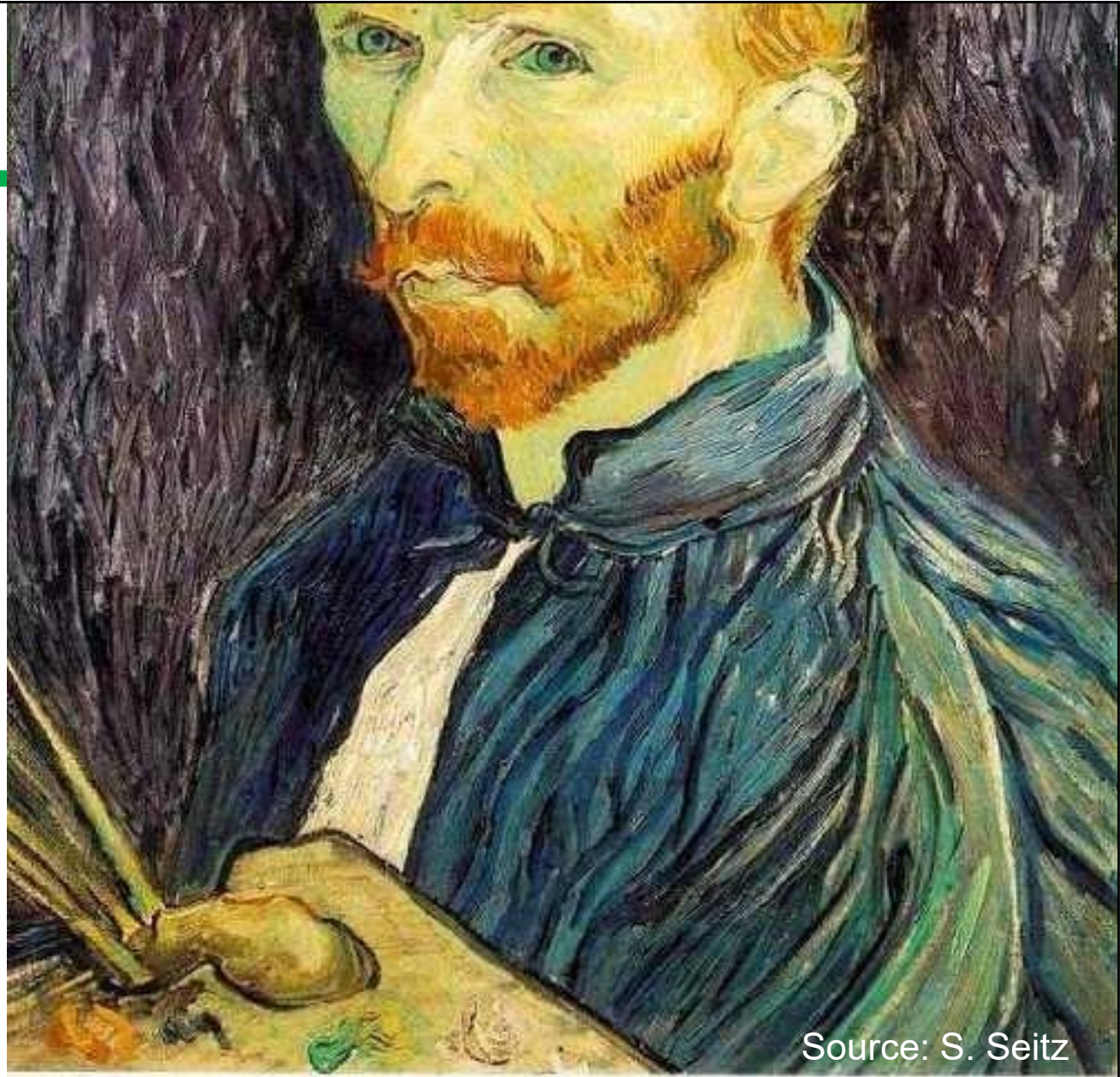
Outline



- Aliasing
- Image Pyramid
- Interpolation
- Related Codes

Image scaling

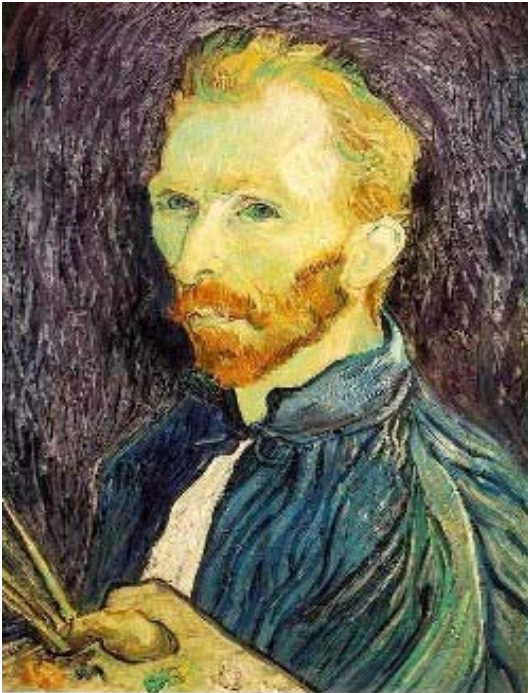
This image is too big to fit on the screen. How can we generate a half-sized version?



Source: S. Seitz

Image Down-sampling

- Throw away every other row and column to create a $1/2$ size image
 - called *image sub-sampling*



Original size



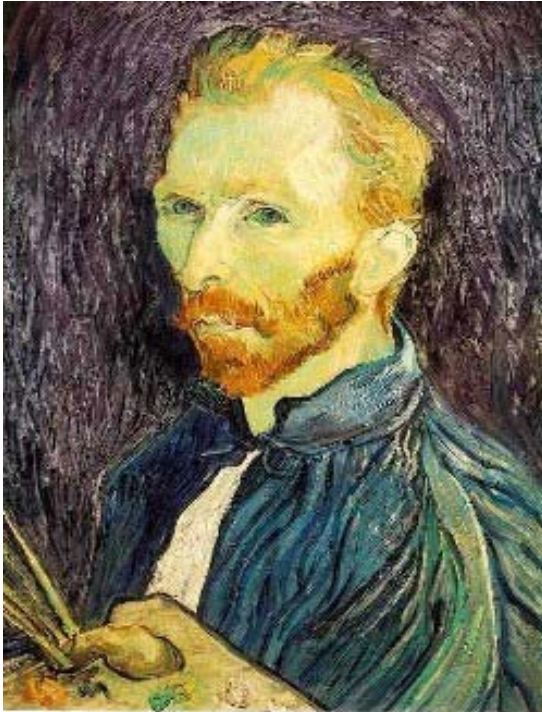
$1/4$



$1/8$

Image Down-sampling

- Down sampled image is so pixelated



1/2



1/4 (2x zoom)



1/8 (4x zoom)

Image Down-sampling

- Another down-sampling images

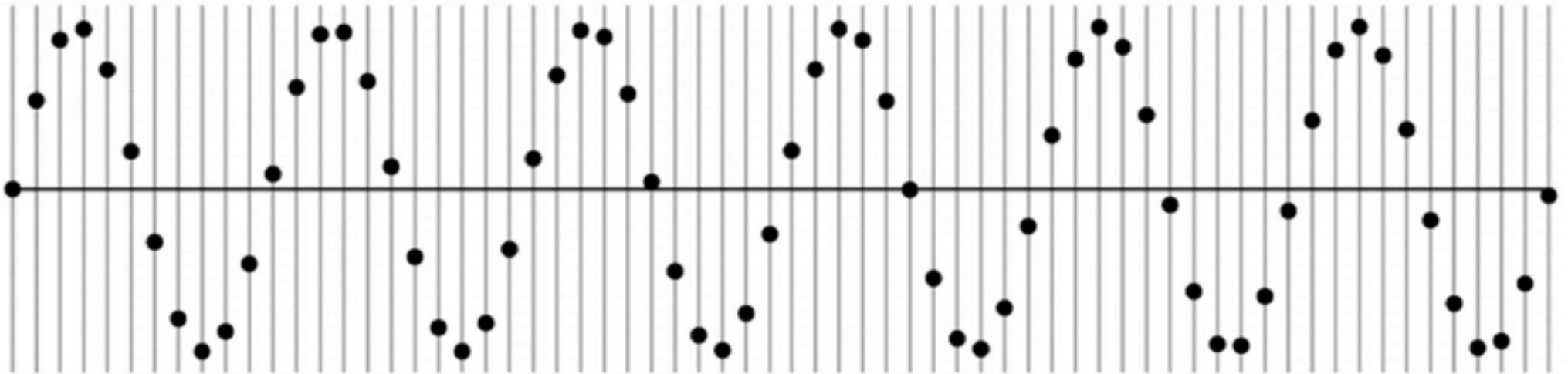


Source: S. Seitz

Aliasing

Sampling

- Ex) sine wave



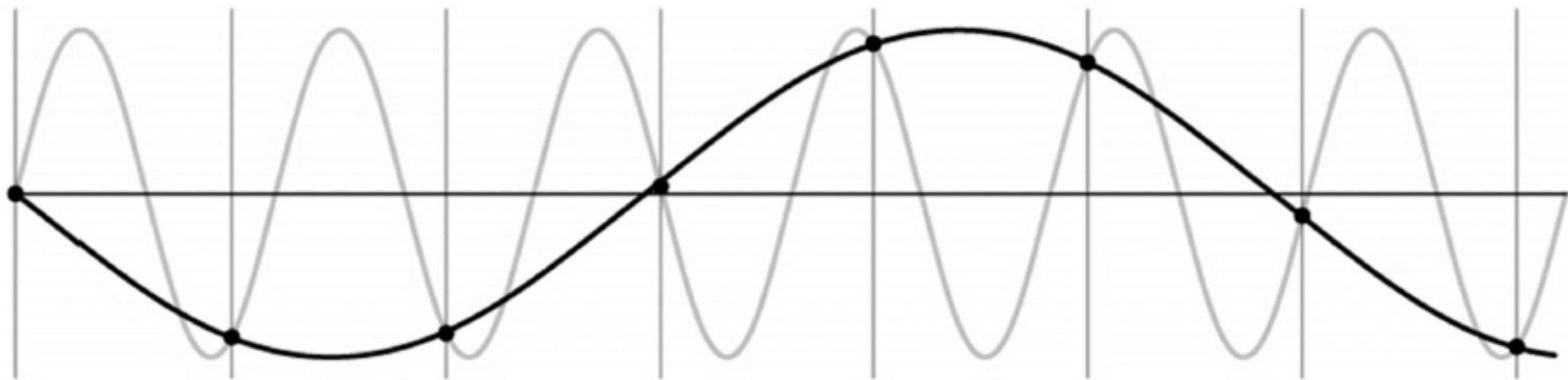
Sampling

- Ex) sine wave



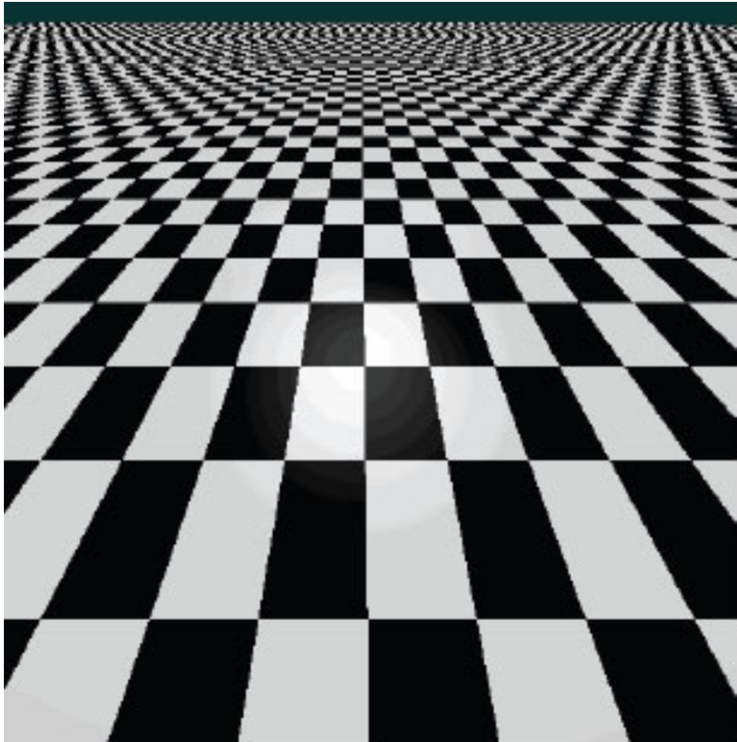
Aliasing

- Occurs when your sampling rate is not high enough to capture the amount of detail in your image
- Can give you the wrong signal/image—an *alias*



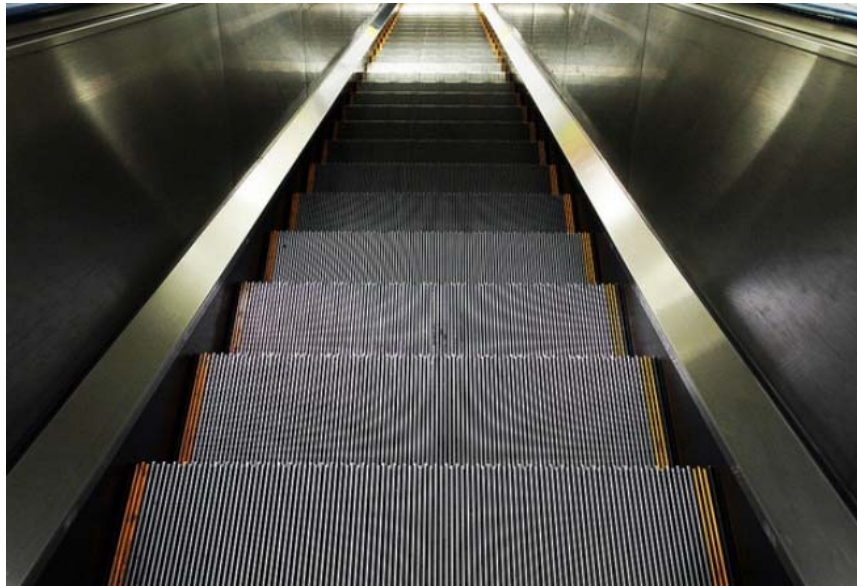
Aliasing

- Texture



Aliasing

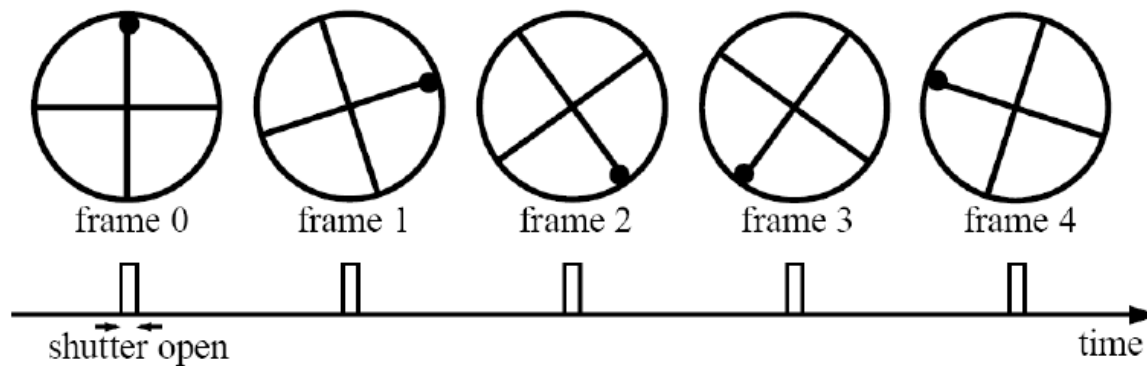
- Photograph



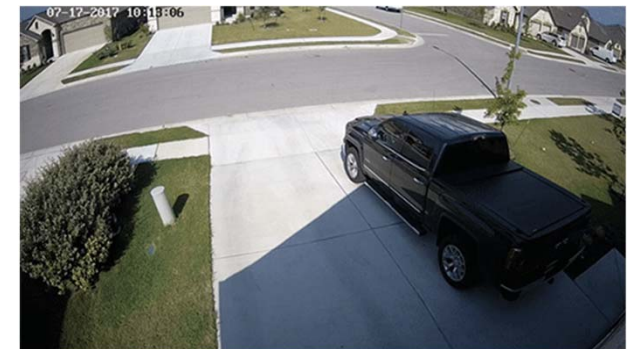
Wagon-wheel effect

Imagine a spoked wheel moving to the right (rotating clockwise).
Mark wheel with dot so we can see what's happening.

If camera shutter is only open for a fraction of a frame time (frame time = $1/30$ sec. for video, $1/24$ sec. for film):



Without dot, wheel appears to be rotating slowly backwards!
(counterclockwise)



Wagon-wheel effect

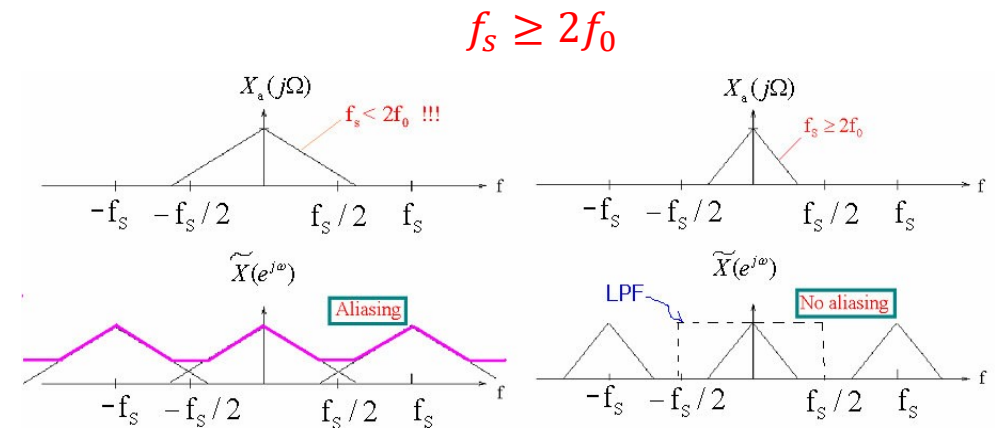


Aliasing

- To avoid aliasing:

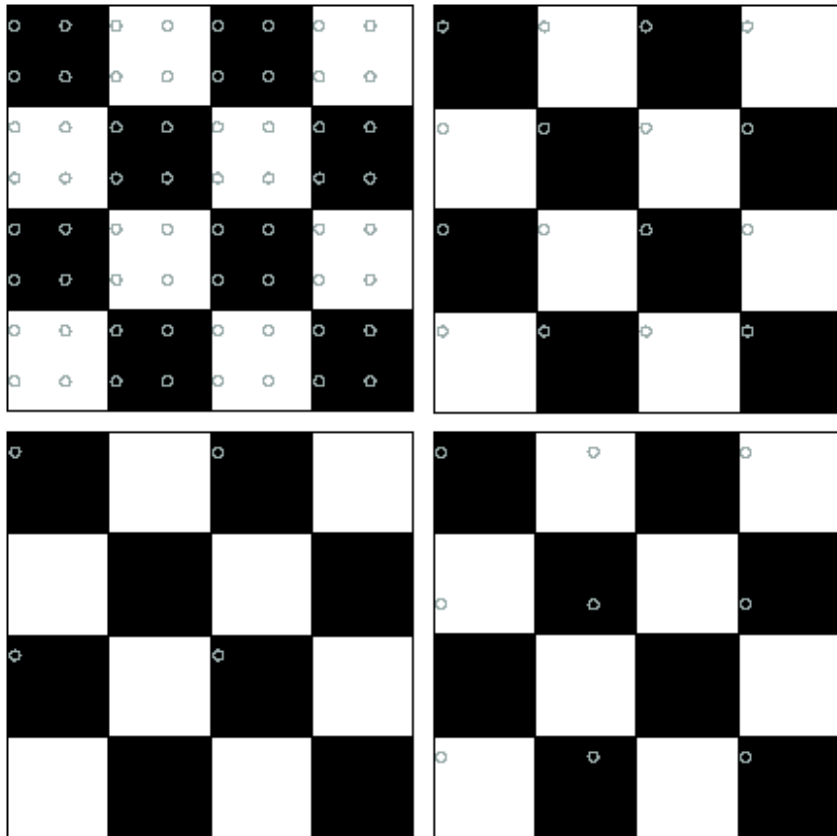
Solution 1) Oversample the signal

- sampling rate $\geq 2 \cdot \text{max frequency in the image}$
 - said another way: $\geq$ two samples per cycle
- This minimum sampling rate is called the Nyquist rate



Aliasing

■ Nyquist limit – 2D example



Good sampling

- When downsampling by a factor of two
 - Original image has frequencies that are too high
- How can we fix this?

Bad sampling

Aliasing



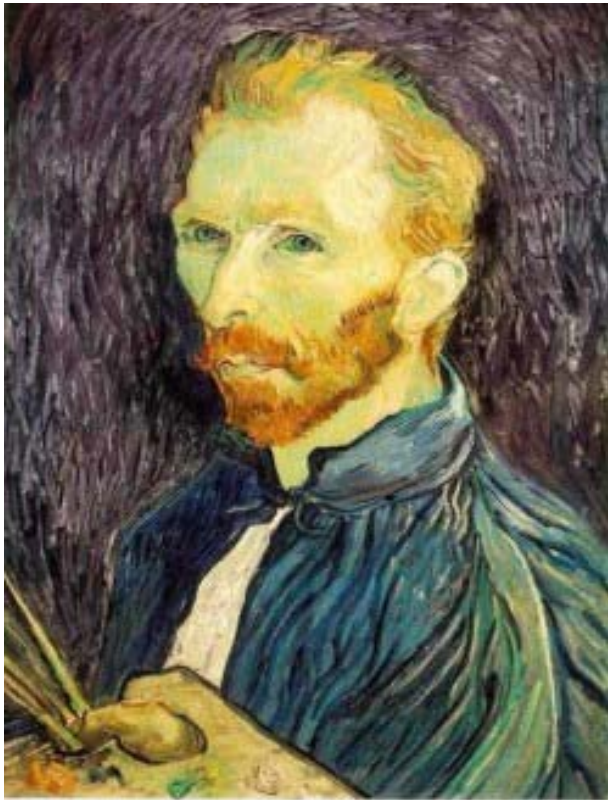
- To avoid aliasing:

Solution 2) filter the image, *then* subsample

- Remove some of the detail effects that cause aliasing
- Lose information, but better than aliasing artifacts

Aliasing

- Apply a smoothing filter first, then throw away half the rows and columns



Gaussian 1/2



G 1/4

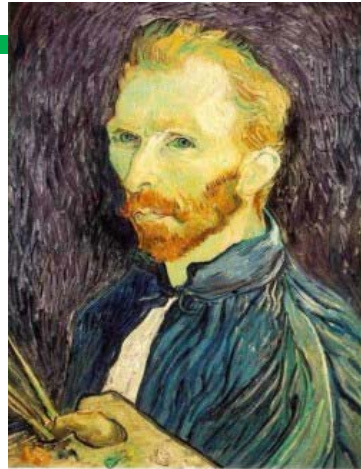


G 1/8

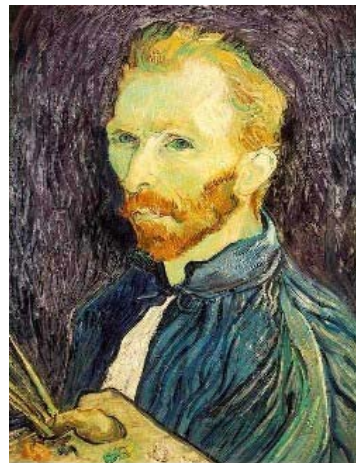
Aliasing

- Compare with...

Original size

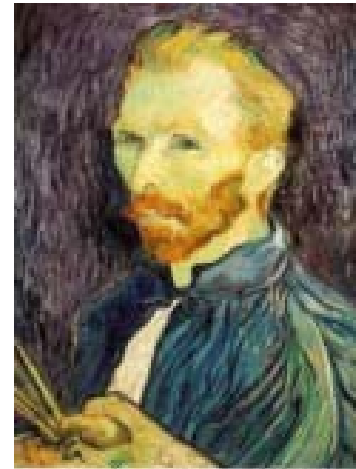


Gaussian 1/2



1/2

(2x zoom)



G 1/4



1/4

(4x zoom)



G 1/8

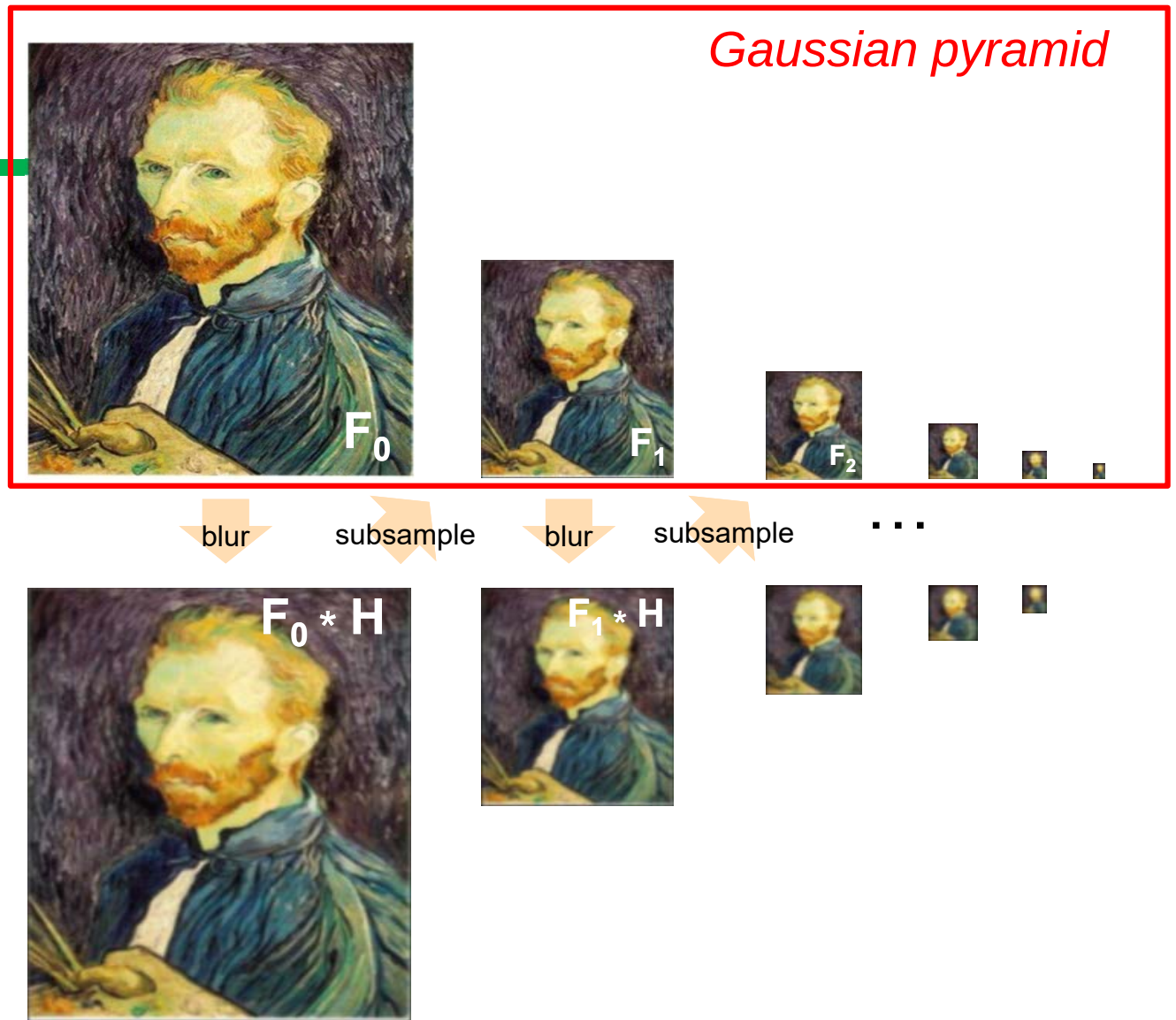


1/8

Gaussian Image pyramid

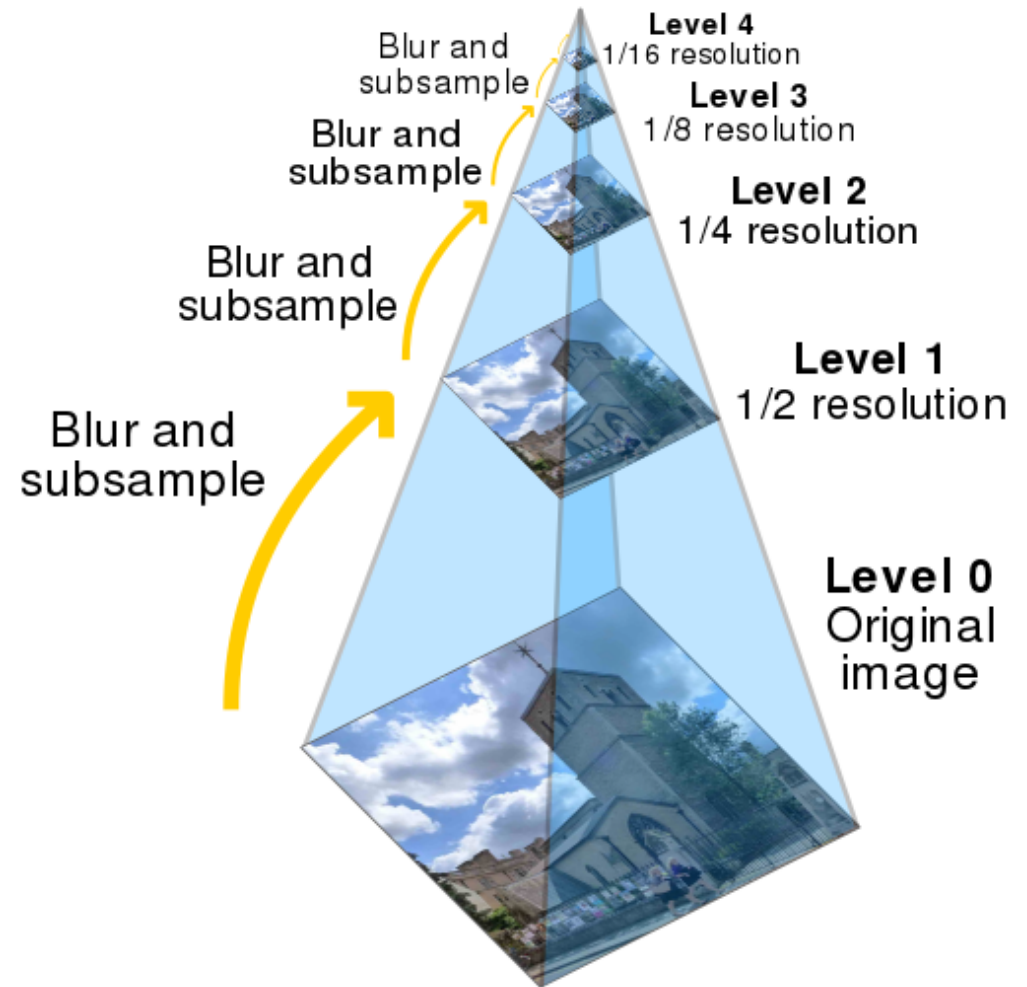
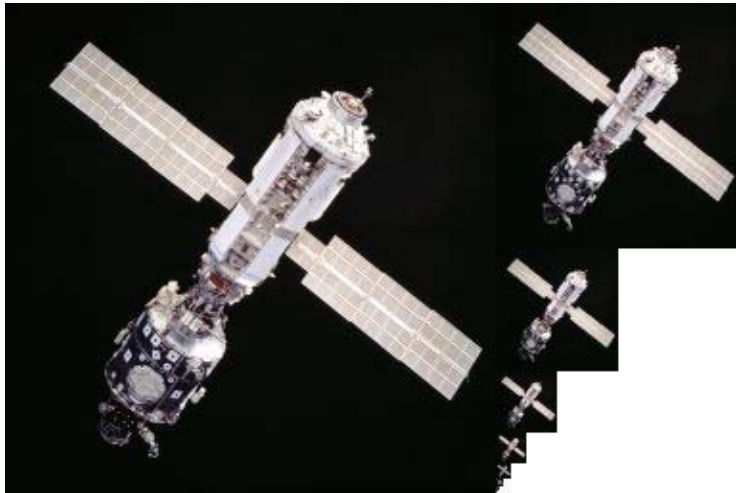
Gaussian Pyramid

- filter the image,
then subsample



Gaussian Pyramid

How much space does a Gaussian pyramid take compared to the original image?

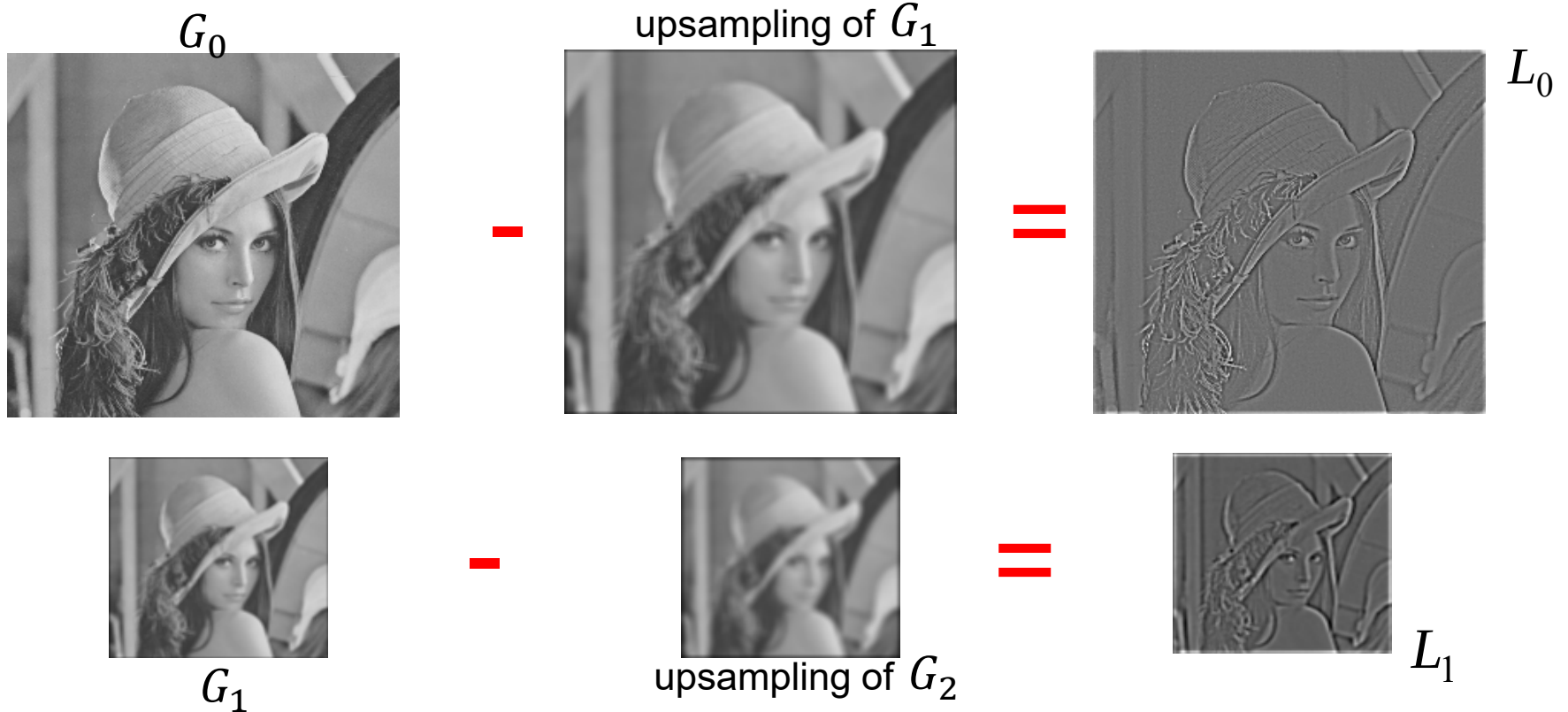


Gaussian Pyramid

- Some Properties of Gaussian Image Pyramid
 - Gaussian Pyramids have all sorts of applications in computer vision
 - What happens to the details of the image?
 - They get smoothed out as we move to higher levels.
 - What is preserved at the higher levels?
 - Mostly large uniform regions in the original image.
 - How would you reconstruct the original image from the image at the upper level?
 - Impossible.

Gaussian Pyramid

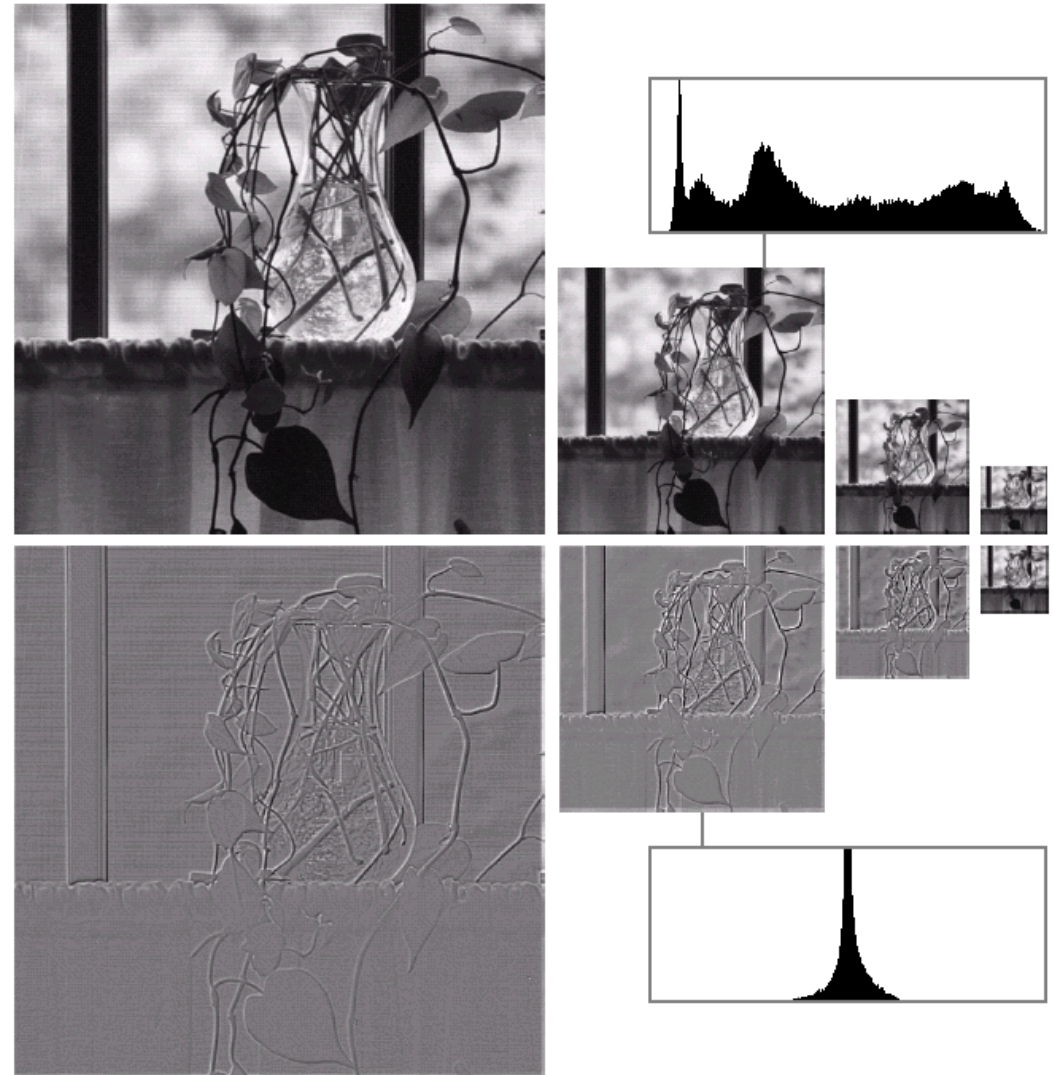
- Blurring makes lossy image
- Is there a way to use residual?



Laplacian Image pyramid

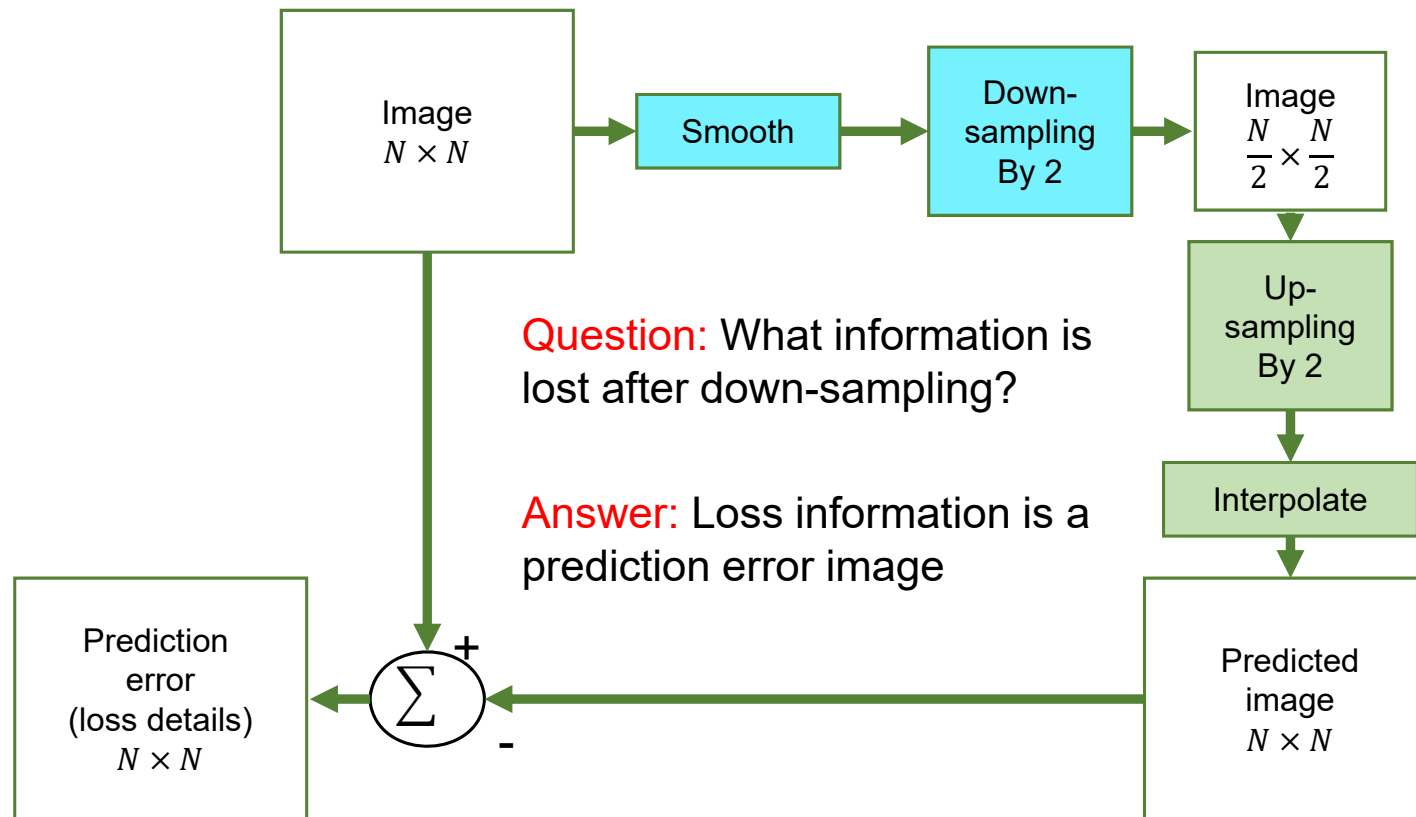
Laplacian Pyramid

- What do we need to store for the reconstruction?
 - At each level, retain the residuals instead of the blurred images themselves.



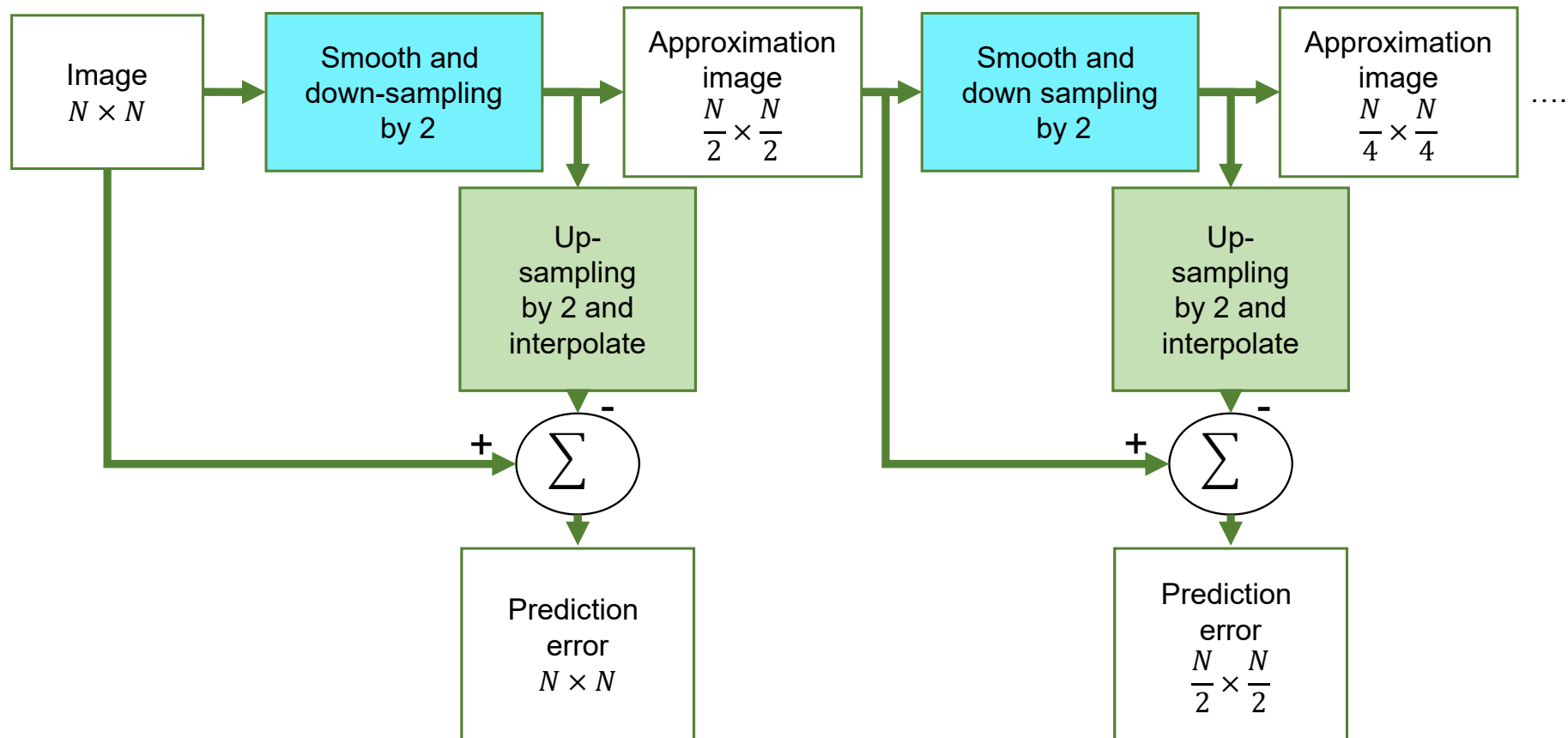
Laplacian Pyramid

- Multiscale decomposition



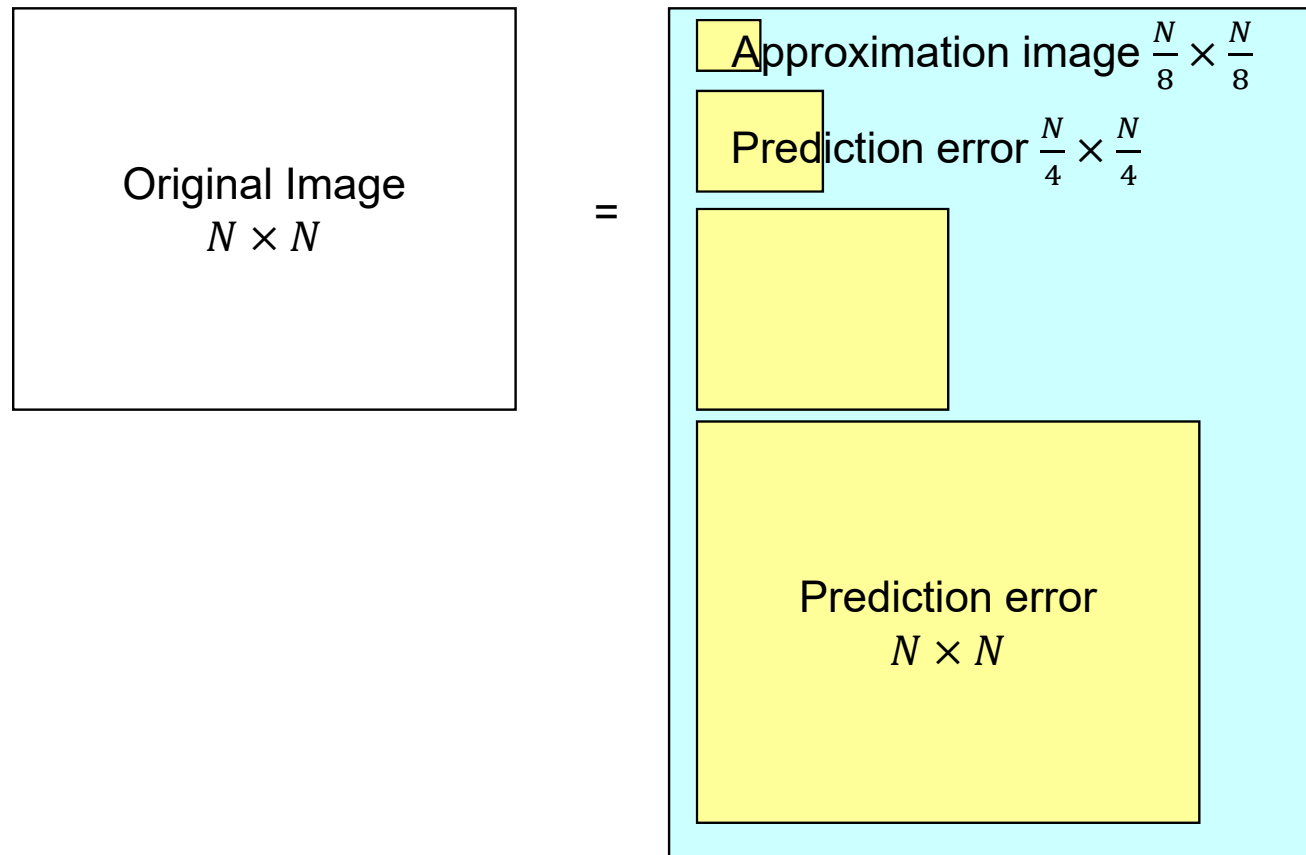
Laplacian Pyramid

- Multiscale decomposition



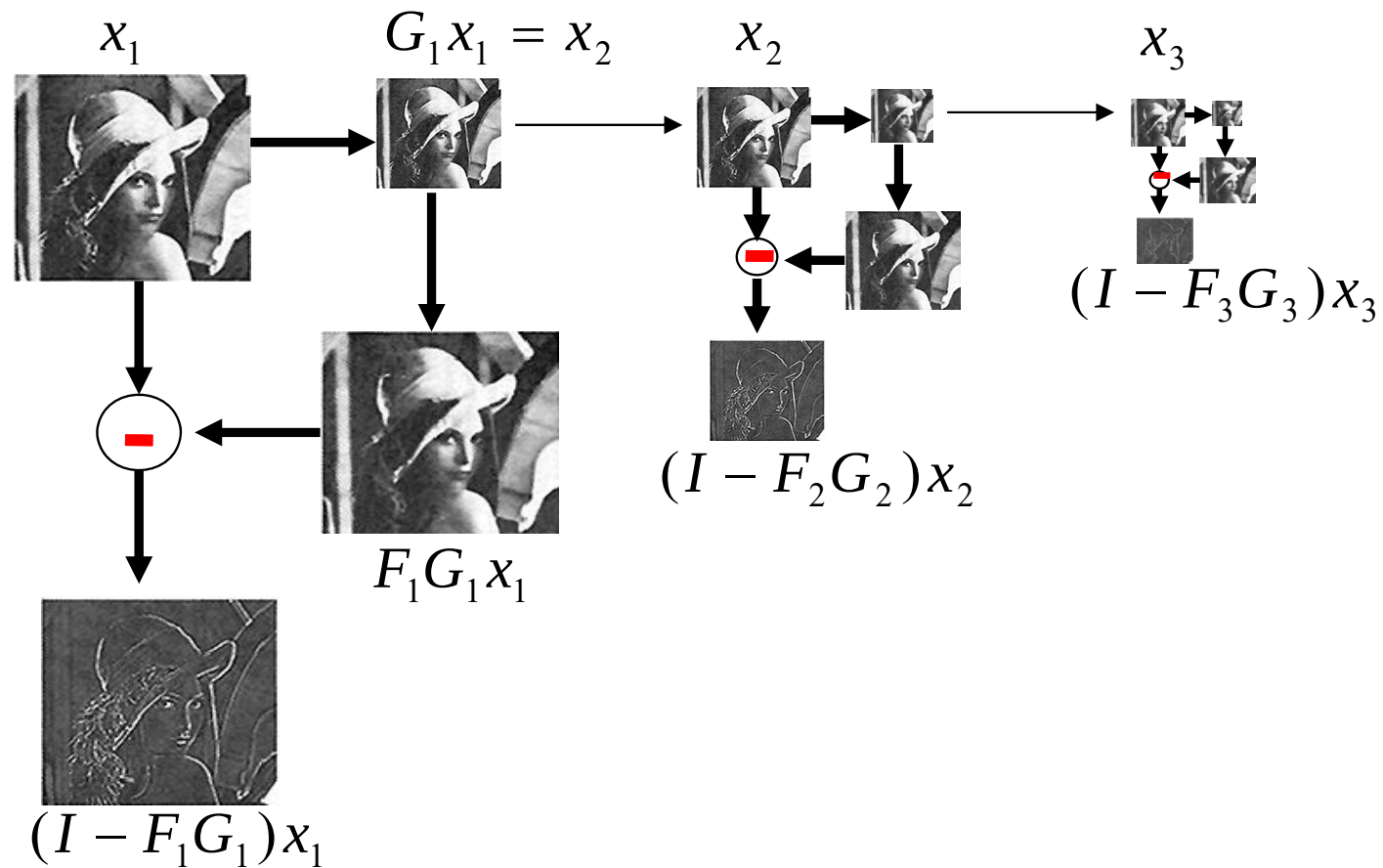
Laplacian Pyramid

- Multiscale decomposition



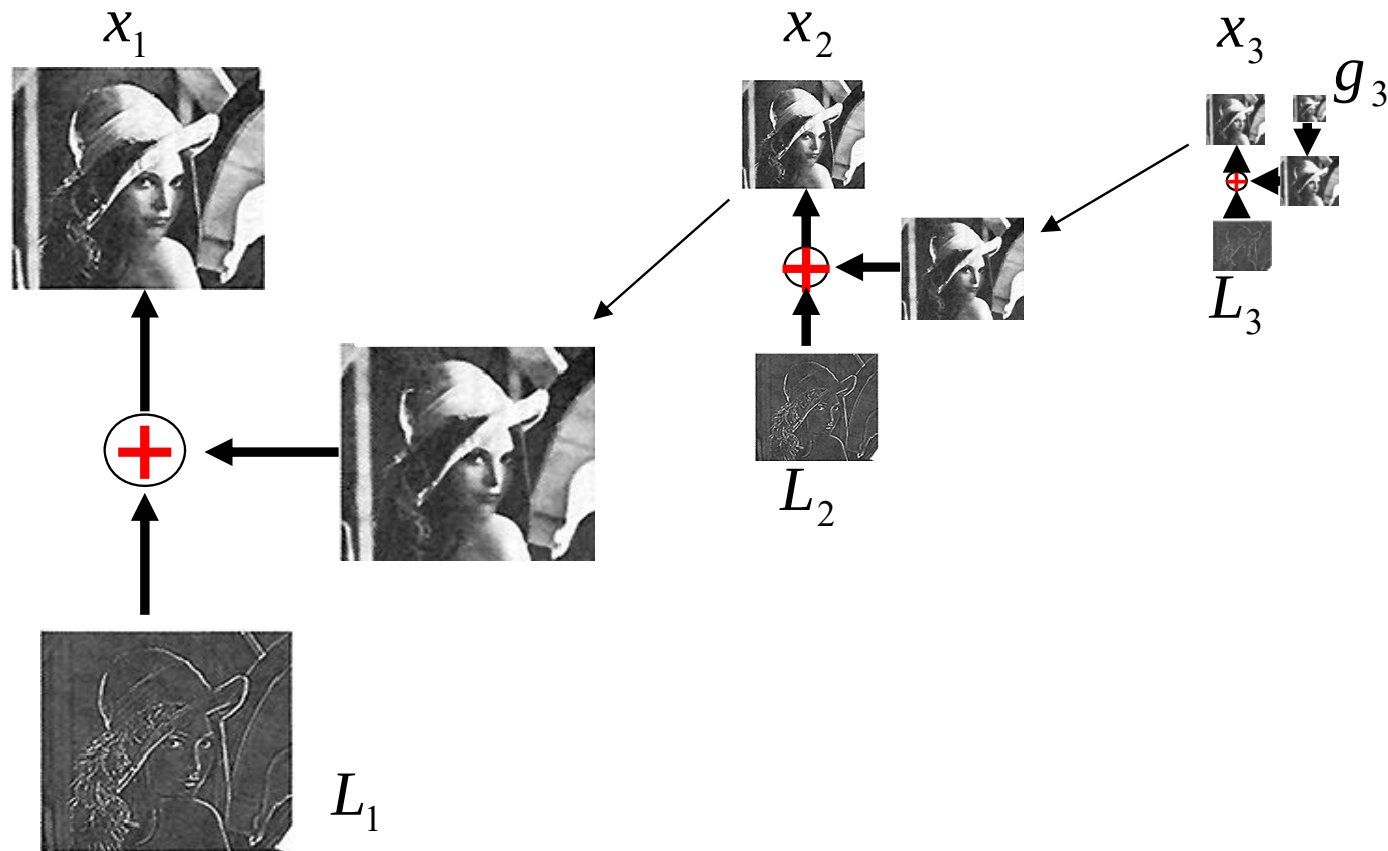
Laplacian Pyramid

- Compute residuals



Laplacian Pyramid

- Reconstruction: Recover x_1 from L_1, L_2, L_3 and g_3



Laplacian Pyramid

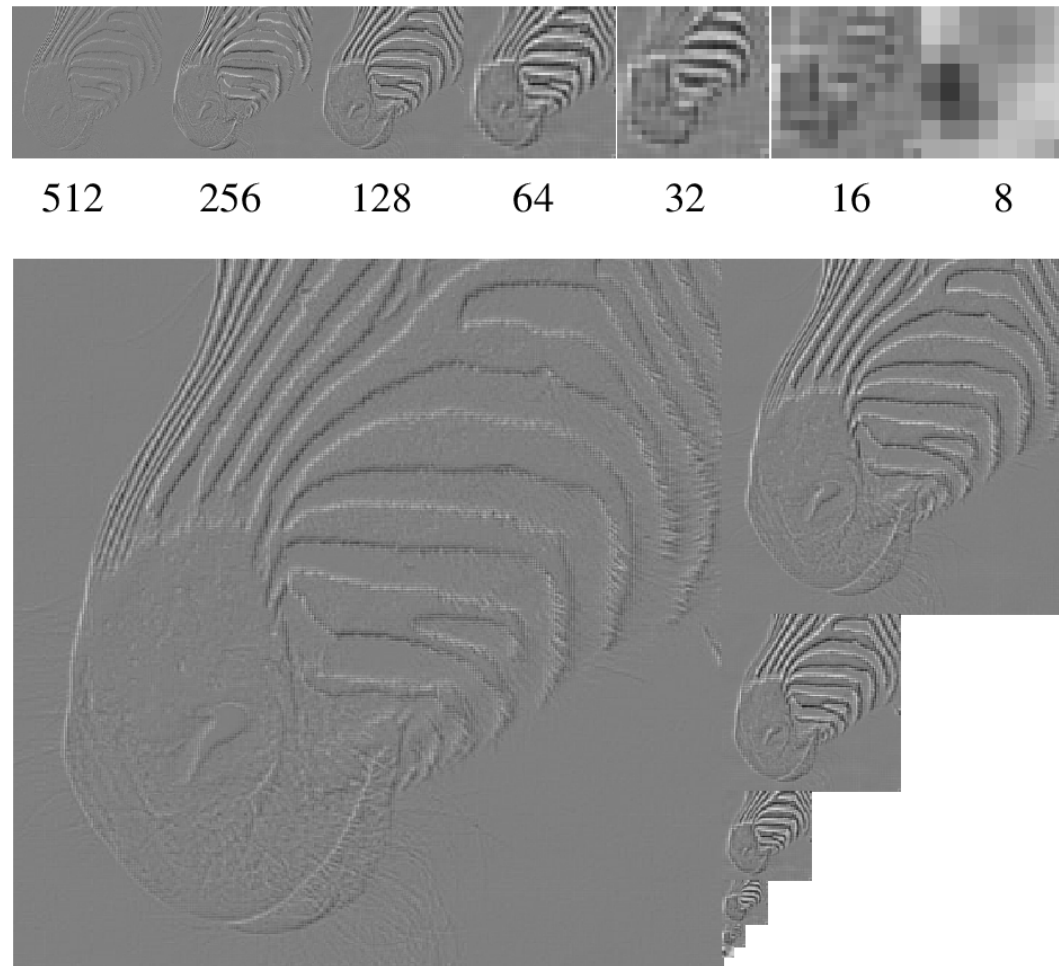


Image Pyramid

- Gaussian pyramid
 - Progressively blurred and subsampled versions of the image
 - Adds scale invariance to fixed-size algorithms
- Laplacian pyramid
 - Shows the information added in Gaussian pyramid at each spatial scale
 - Useful for noise reduction and coding

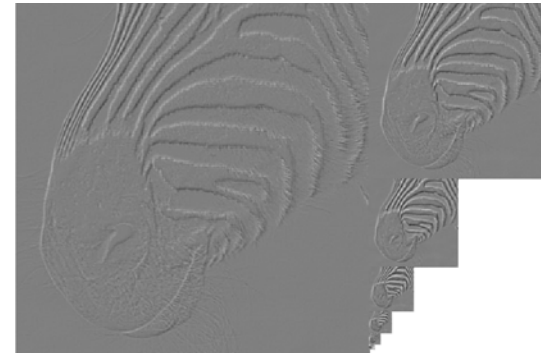


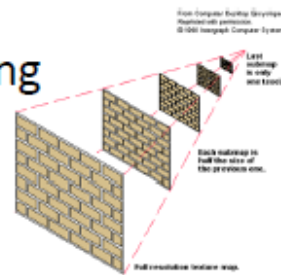
Image Pyramid

- Applications of Image Pyramids

image compression



multi-scale
texture mapping



denoising



focal stack compositing



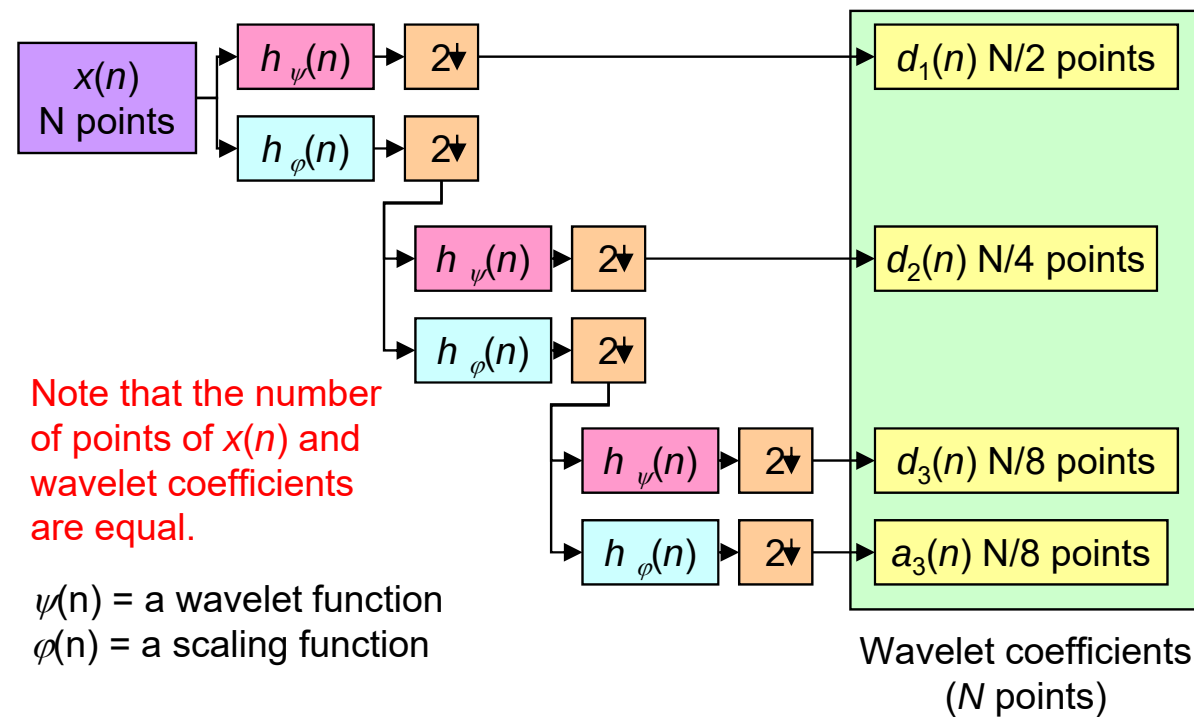
multi-scale image analysis



Wavelet Transform

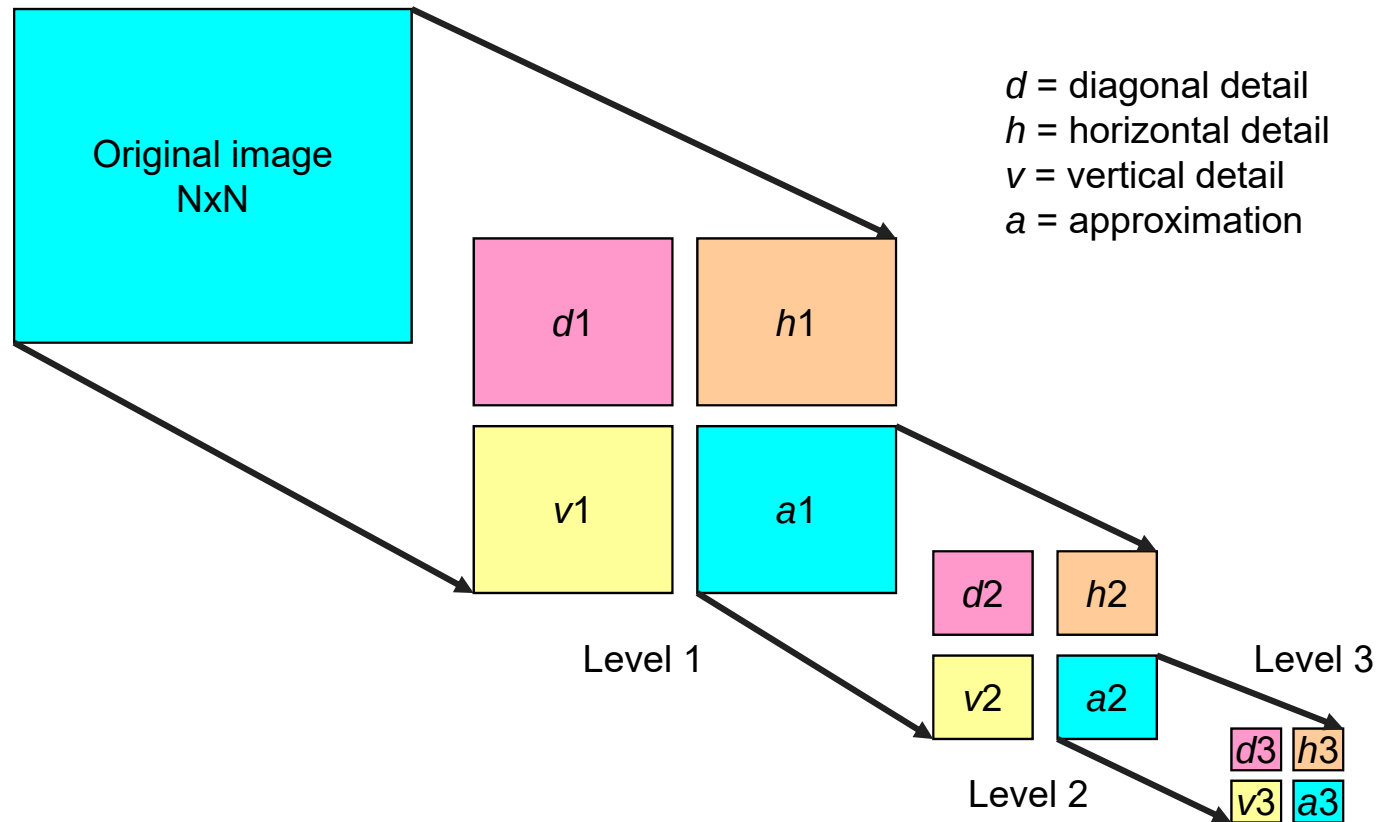
Wavelet Transform

- Wavelet coding is a variant of discrete cosine transform (DCT) coding that uses wavelets instead of DCT's block-based algorithm
- 1D discrete wavelet transformation



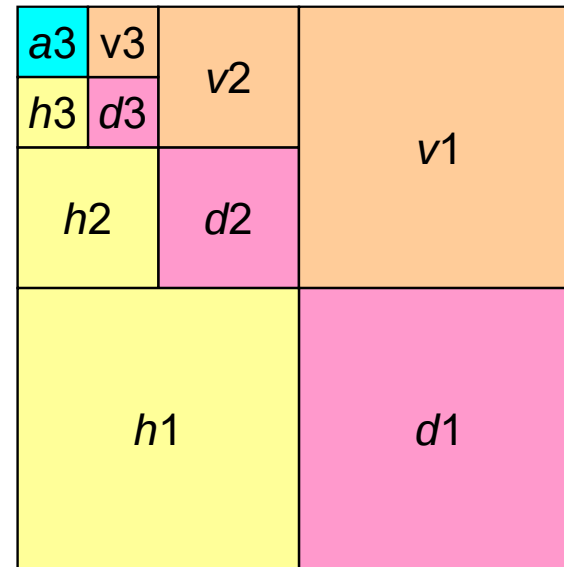
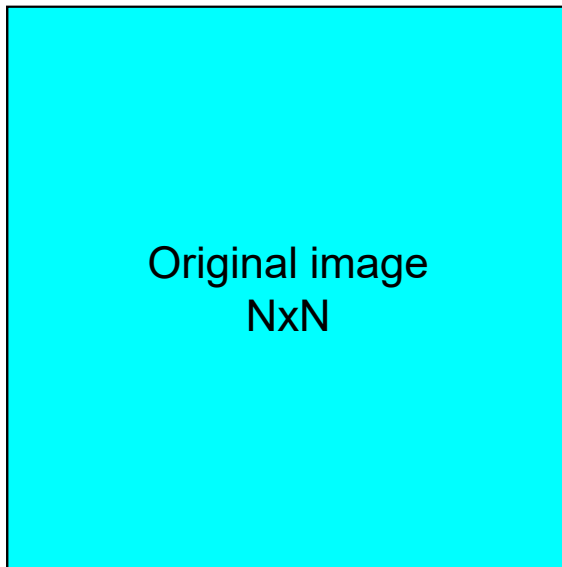
Wavelet Transform

- 2D discrete wavelet transformation



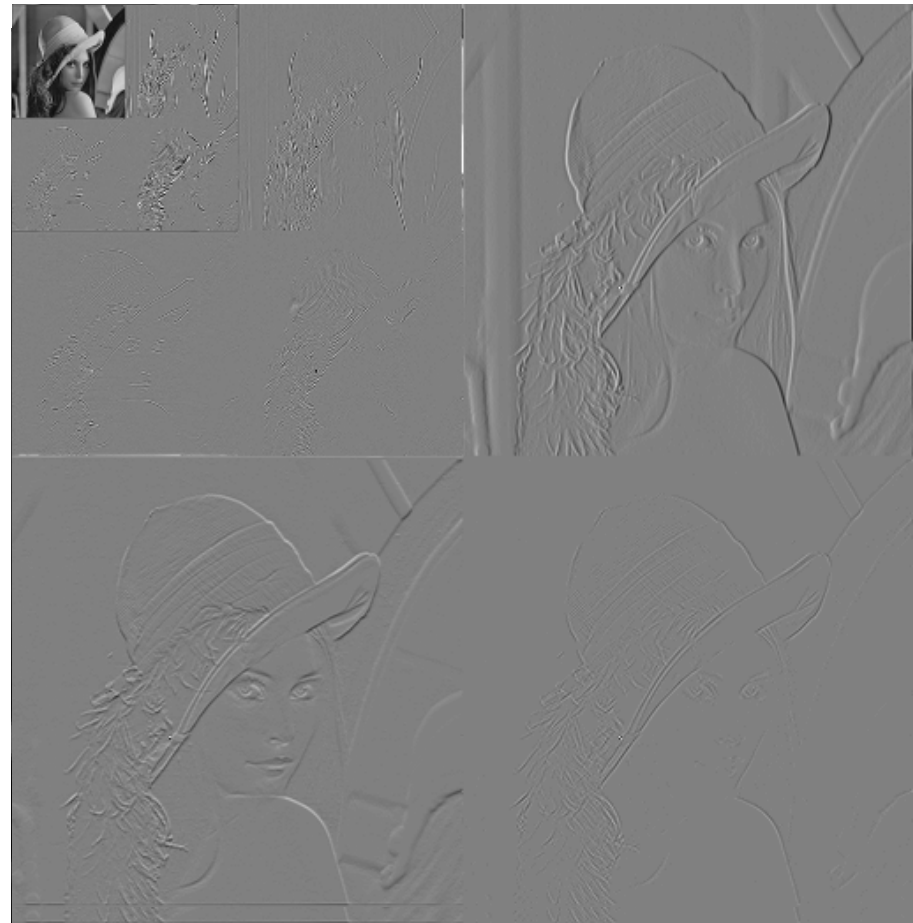
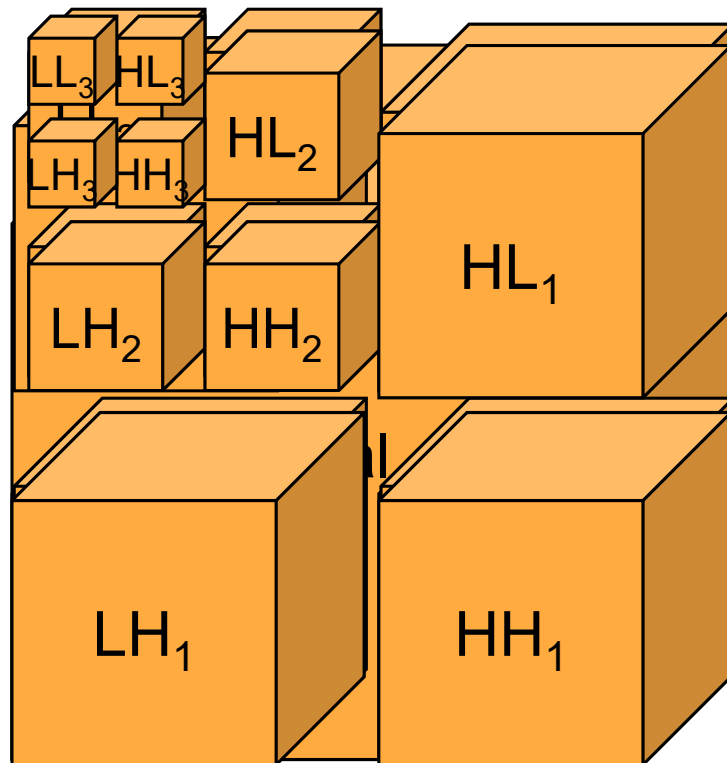
Wavelet Transform

- 2D discrete wavelet transformation



Wavelet Transform

- Example (step)



Wavelet Transform

- Example (step)



Level 1



Level 2

Interpolation

Upsampling

- This image is too small for this screen:
- How can we make it 10 times as big?
- Simplest approach:
 - repeat each row
 - and column 10 times
- (“Nearest neighbor interpolation”)

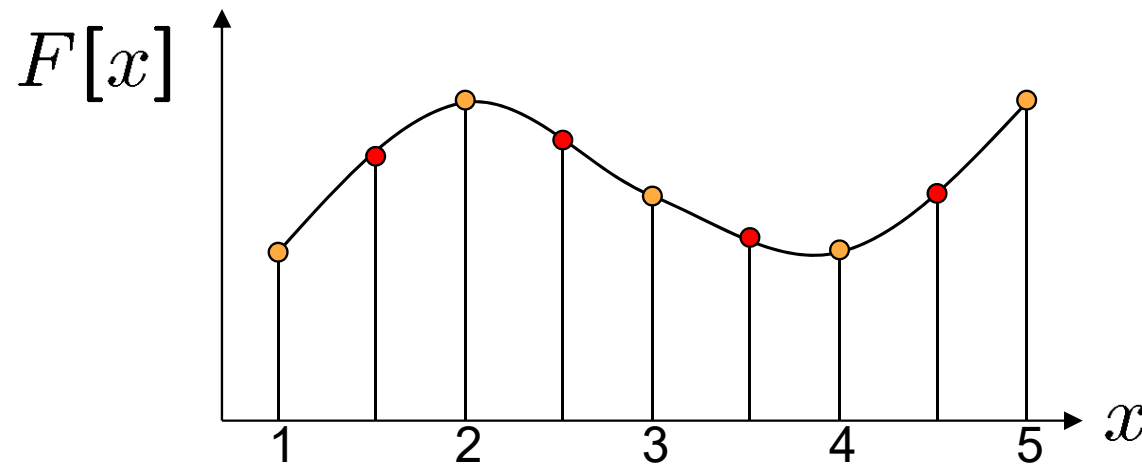


Image interpolation

- Recall that a digital images is formed as follows:

$$F[x, y] = \text{quantize}\{f(xd, yd)\}$$

- It is a discrete point-sampling of a continuous function
- If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale



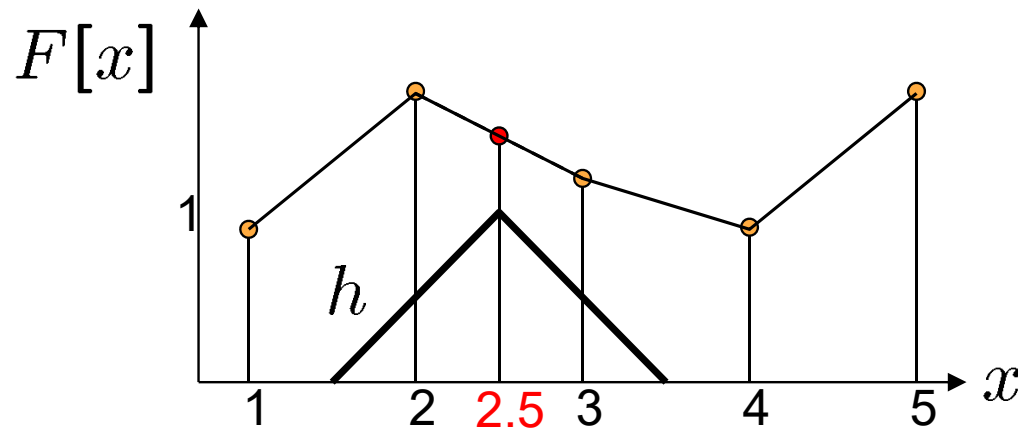
$d = 1$ in this example

Image interpolation

- What if we don't know f ?
 - Guess an approximation: $\tilde{f}$
 - Can be done in a principled way: filtering
 - Convert F to a continuous function:

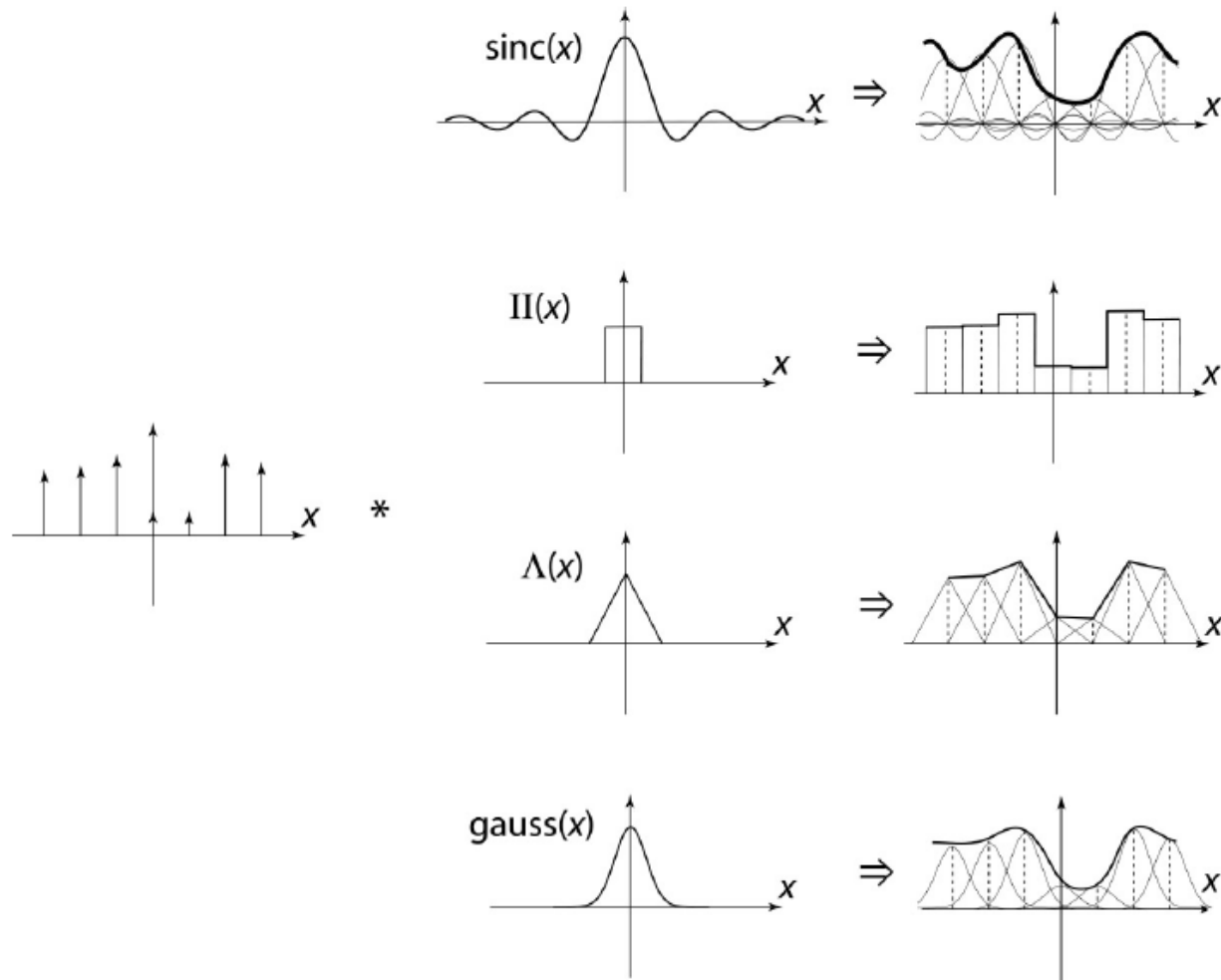
$$f_F(x) = F\left(\frac{x}{d}\right) \text{ when } \frac{x}{d} \text{ is an integer, } 0 \text{ otherwise}$$

- Reconstruct by convolution with a *reconstruction filter*, h $\tilde{f} = h * f_F$



$d = 1$ in this example

Image interpolation



"Ideal" reconstruction

Nearest-neighbor interpolation

Linear interpolation

Gaussian reconstruction

Image interpolation

- Most common interpolation techniques :

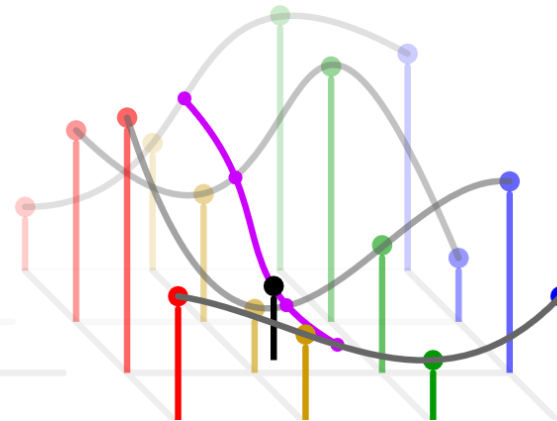
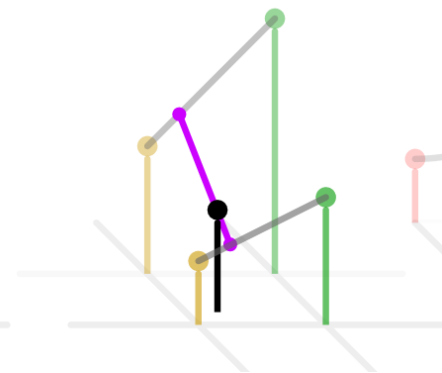
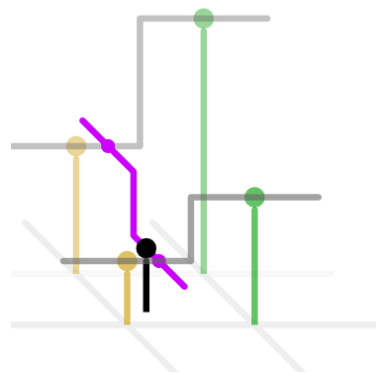
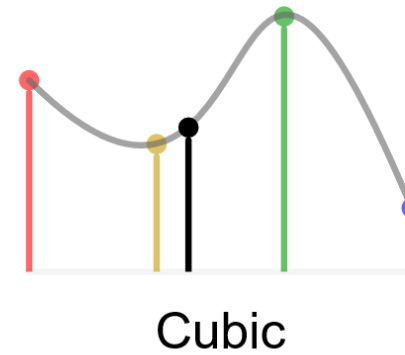
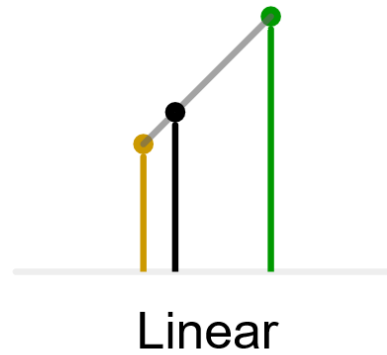
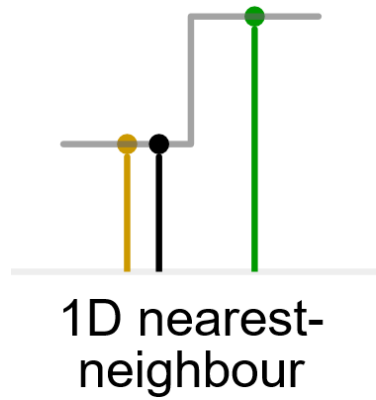


Image interpolation

■ Bilinear Interpolation

- produces a smoother interpolation than does the nearest neighbor approach
- is used when we need to know values at random position on a regular 2D grid. Note that this grid can as well be an image or a texture map
- 1D linear interpolation

$$f(x) = \frac{d_2}{d_1 + d_2} f(x_1) + \frac{d_1}{d_1 + d_2} f(x_2)$$

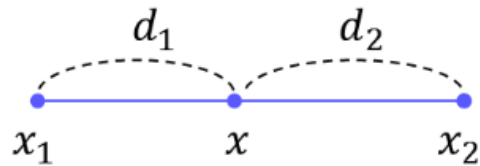


Image interpolation

- Bilinear Interpolation

$$f(x, y_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}),$$

$$f(x, y_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}).$$

$$\begin{aligned} f(x, y) &\approx \frac{y_2 - y}{y_2 - y_1} f(x, y_1) + \frac{y - y_1}{y_2 - y_1} f(x, y_2) \\ &= \frac{y_2 - y}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \right) + \frac{y - y_1}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \right) \\ &= \frac{1}{(x_2 - x_1)(y_2 - y_1)} (f(Q_{11})(x_2 - x)(y_2 - y) + f(Q_{21})(x - x_1)(y_2 - y) + f(Q_{12})(x_2 - x)(y - y_1) + f(Q_{22})(x - x_1)(y - y_1)) \\ &= \frac{1}{(x_2 - x_1)(y_2 - y_1)} \begin{bmatrix} x_2 - x & x - x_1 \end{bmatrix} \begin{bmatrix} f(Q_{11}) & f(Q_{12}) \\ f(Q_{21}) & f(Q_{22}) \end{bmatrix} \begin{bmatrix} y_2 - y \\ y - y_1 \end{bmatrix}. \end{aligned}$$

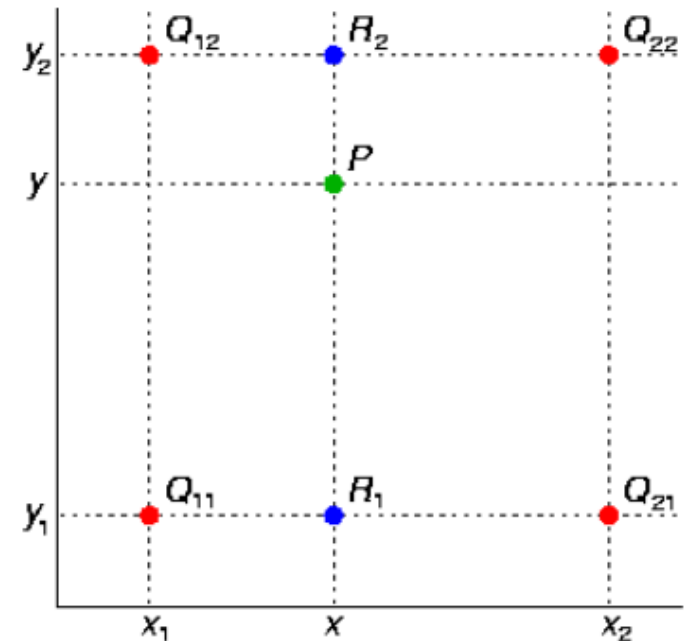


Image interpolation

- Bilinear Interpolation

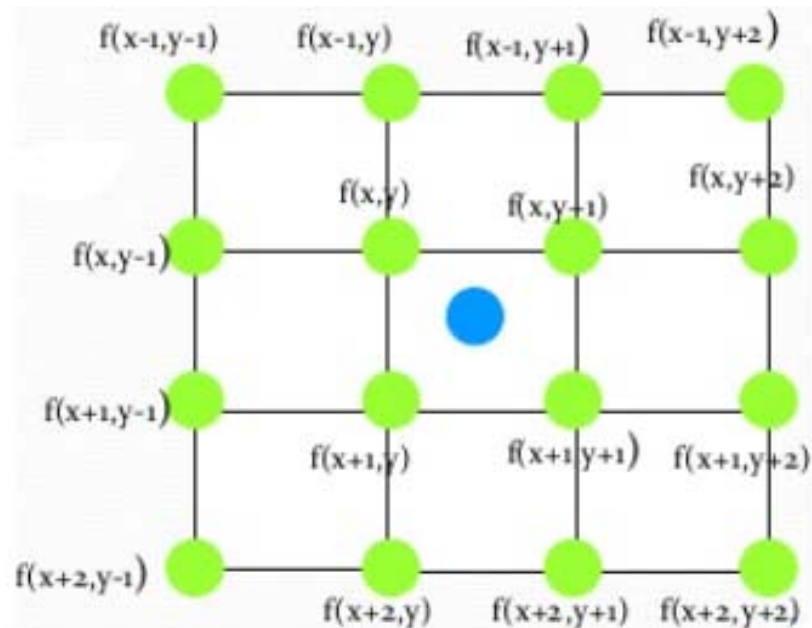
10	15	24
24	18	27
16	21	30



10			15			24
24			18			27
					?	
16			21			30

Image interpolation

- Bicubic Interpolation



$$g(x, y) = \frac{1}{16} \sum_{l=-1}^2 \sum_{m=-1}^2 f(x + l, y + m) u(dx) u(dy)$$

$f(x + l, y + m)$: value of pixel

$u(dx)$: variation on x-axis

$u(dy)$: variation on y-axis

Image interpolation



Nearest-neighbor interpolation



Bilinear interpolation



Bicubic interpolation

Image interpolation

Original image:  x 10



Nearest-neighbor interpolation



Bilinear interpolation



Bicubic interpolation

Related Codes

■ Image Pyramid

- **Gaussian** Pyramid and **Laplacian** Pyramids

https://docs.opencv.org/master/dc/dff/tutorial_py_pyramids.html

■ Smoothing Images - 2D Convolution(Image Filtering)

- Gaussian Filtering :

https://docs.opencv.org/master/d4/d13/tutorial_py_filtering.html

■ Youtube

- Image Pyramids with Python and OpenCV

<https://youtu.be/8yvln2atFkA>

- Image Blending using Pyramids

<https://youtu.be/HfM9s2ehErE>

Thanks!